CPSC 57700 Machine Learning Operations

Fall 2026 -Week 1 Paper

INFO

Name	Neelu Tiwari
Email	NeeluVidyashankarT@lewisu.edu
ID	L30173558

ACM Article: "We Have No Idea How Models Will Behave in Production until Production: How Engineers Operationalize Machine Learning"

Q1. Challenges in MLOps

- Maintaining Good Data Quality: One of the main challenges in MLOps is maintaining good data quality, especially when working with large amounts of data. Machine learning models learn from examples, so the quality of those examples affects the model's predictions. If the data contains incorrect labels or inconsistent information, the model may learn the wrong patterns.

  For example, a team may develop a model that identifies whether an email is spam. If different people label similar emails differently, the training data may contain conflicting answers. The model may then have difficulty learning which emails should be classified as spam. This problem becomes harder when an organization uses many people or outside teams to label large amounts of data.

  The article explains that checking labels becomes more difficult as the amount of data increases. Humans may need to examine unclear examples and correct mistakes. Without proper checking, incorrect labels can become part of the training process and reduce the quality of future models. Therefore, organizations need to pay attention to data quality instead of assuming that having more data will always produce better results.

- Difficulty in Finding Model Errors

  Machine learning errors are not always easy to identify. In traditional software, a problem may cause an application to crash and produce an obvious error message. A machine learning model can continue running normally while producing poor predictions.

  For example, a model may use information during training that would not actually be available when making a real prediction. **This is called data leakage**. The model may appear highly accurate during testing but perform poorly after deployment. Other problems are caused by random settings during training. These errors may not be discovered through ordinary software tests because the program can still run without

crashing. Engineers may examine different groups of data, compare results, and investigate unusual predictions to find the cause.

The article shows that testing machine learning systems requires more than checking whether the code works. Engineers must also examine the data, the model's behaviour, and the results produced in real situations.

➢ Delayed Feedback:

In a production system, the actual outcomes of a prediction may not be available immediately. This makes it more difficult for engineers to know whether the model is still performing well or when it needs to be retrained. Unlike a test dataset with known answers, real-world results may take time to confirm.

➢ Too Many Alerts:

Engineers use data checks and alerts to identify missing values and unexpected changes. However, normal changes in real-world data can also trigger alerts. If engineers receive too many false alarms, they may start ignoring notifications and miss a genuine problem.

➢ Temporary fixes:

When a model produces incorrect predictions, engineers may add rules or filters to address the immediate problem instead of retraining the model. Although this can be a quick solution, too many temporary fixes can make the pipeline complicated and difficult to maintain.

Q2. Author's solutions to address these challenges?

The authors discuss three key ideas for improving MLOps: Visibility, Versioning, and Velocity. These ideas help engineering teams work efficiently while keeping their system reliable.

➢ Velocity: Making Experiments Faster and Safer:

Velocity refers to how quickly engineers can test ideas, make changes, and improve a model. Machine learning development often involves trying different features, settings, and data preparation methods. Teams need to perform these experiments without making the entire system difficult to manage.

The authors explain that engineers should make small controlled changes rather than changing many parts of the system at once. They can also keep settings in separate configuration files, such as JSON or YAML files, instead of repeatedly modifying the main application code.

Another solution is to deploy changes gradually; instead of sending a new model to every user immediately, a team can first test it in a limited environment and then increase its use after checking the results. This reduces the risk of a problem affecting the entire product.

These practices help teams experiment quickly while keeping changes manageable. They also make it easier to identify which change caused a problem if something goes wrong.

➢ Visibility:

Visibility means having enough information to understand how a machine learning system behaves after deployment. Since model problems may not cause an application to crash, engineers need logs, measurements, and other monitoring tools to identify unusual behaviour.

The authors recommend collecting information throughout the processing pipeline. Engineers should be able to check incoming data, identify missing information, review model results, and investigate changes in performance. Visual tools can also help them compare results across different groups of users or types of data.

Visibility is not limited to technical teams. Engineers also need to communicate with business managers and other people involved in the project. A model may perform well according to a technical measurement but fail to improve the business outcome that the organization actually cares about.

For example, a recommendation model may produce technically reasonable recommendations, but the company may also need to check whether customers are clicking on them or finding them useful. Connecting technical results with business goals helps teams decide whether a model change is worth keeping.

➢ Versioning: Keeping Track of Changes

Versioning means keeping organized records of the different parts of a machine learning system. These include the data, application code, trained models, and additional rules used to process predictions.

This is important because a machine learning system depends on several connected components. If an engineer changes the training data or modifies a rule, the results may change even when the model's main code remains the same. Without proper records, it can be difficult to understand why performance improved or declined.

The authors explain that keeping versions of these components allows teams to return to an earlier, reliable model when a new version causes problems. They can also use a simpler backup approach to maintain the service while investigating the issue.

Keeping records of unsuccessful experiments is useful as well. It helps engineers understand what has already been tried and prevents teams from repeating the same mistakes. It also makes it easier for new team members to understand previous decisions.

Together, velocity, visibility, and versioning provide a practical approach to managing machine learning systems. They help teams make changes carefully, understand problems, and recover when a new version does not perform as expected.

Q3. Various stages in MLOps:

The article describes four main stages in MLOps: data preparation, experimentation, evaluation and deployment, and monitoring and response. These stages are connected and form a continuous process. Engineers may need to return to an earlier stage when they discover a problem or find a way to improve the model.

➢ Stage one: Data preparation:

Data preparation is the first stage. It involves collecting data, cleaning it, checking its quality, and preparing it for model training. The data must be organized so that it can be used consistently. In a production environment, data preparation often follows scheduled processes.

These processes collect new data and apply the required transformations. Engineers may also create new features, which are the input information used by a model to make predictions.

Checking the quality of labels is another important part of this stage. Human reviewers may need to examine examples, resolve disagreements, and correct incorrect labels. Automated checkers can identify missing values, unexpected formats, or other problems before the data is used for training.

Good data preparation reduces the chance of training a model on incorrect or unsuitable information. It also gives the team a more reliable starting point for the next stage.

➢ Stage 2: Experimentation

In the experimentation stage, engineers try approaches to improve the model's results. They may change the input features, adjust settings, or prepare the data in different ways. They then compare the results to determine which approach works better.

The article points out that improvement does not always require creating a completely new model. In many cases, changing the information provided to the model can be more useful than changing the model itself.

For example, a customer prediction model may become more useful when engineers add information about recent customer activity. The team can test whether this information improves the predictions.

Experimentation also involves cooperation with people who understand the business problem. Data analysts and subject-matter experts can help engineers identify useful information and explain patterns that may not be obvious from the data alone.

Teams generally benefit from testing one change at a time and recording the results. This makes it easier to understand what worked and use those findings in later experiments.

➢ Stage 3: Evaluation and Deployment

Evaluation and deployment involve checking whether a model is suitable for real use and introducing it into the production system. According to the article, these activities are closely connected because model evaluation must continue after deployment.

Before releasing a model widely, engineers can test it in different environments. They may begin with development testing, then use a limited release or test that runs alongside the existing system. If the new version performs well, it can gradually be made available to more users.

The team should examine both technical measurements and business results. Technical measurements help determine whether predictions are accurate enough, while business measures show whether the model provides practical value. For example, an organization may check whether a new model improves customer engagement or reduces customer loss.

The authors also describe the use of updated validation data. When engineers discover a new type of failure, they can add examples of that failure to the test dataset. Future model versions can then be checked against those examples to reduce the chance of repeating the same mistakes.

➢ Stage 4: Monitoring and response

Monitoring and response is the stage in which engineers observe the running system and deal with problems. After deployment, the data and conditions may change, and a model that worked well earlier may no longer perform as expected.

Engineering teams use monitoring tools, logs, and alerts to identify unusual behaviour. When a problem appears, the responsible engineer investigates the affected data and tries to find the cause. For example, the problem may come from missing input values, a change in customer behaviour, or an issue in the processing pipeline.

The article explains that teams often assign engineers to take responsibility for production issues during scheduled on-call periods. These engineers review alerts, investigate failures, and document what happened.

If fixing the model immediately is not practical, the team may temporarily use a simpler set of rules or return to a previously tested model. This helps keep the service working while engineers develop and test a permanent fix.

Information collected during monitoring can also be used to improve the next version of the model. New failure examples can be added to the validation data, and the team can return to data preparation or experimentation when needed. This makes MLOps a continuous process rather than a task that ends when a model is deployed.

Reference:

"We Have No Idea How Models will Behave in Production until Production": How Engineers Operationalize Machine Learning."

SHREYA SHANKAR,University of California, Berkeley, USA;
ROLANDO GARCIA , University of California, Berkeley, USA
JOSEPH M. HELLERSTEIN, University of California, Berkeley, USA
ADITYA G. PARAMESWARAN, University of California, Berkeley, USA